Small and Fast Depth Estimation with Few Labels by Learning from a Big Model

CS466 Deep Learning, Fall 2026 — Term Project Proposal (Models Track)

Oğuz Kağan Göçmez
Özyeğin University
kagan.gocmez@ozu.edu.tr

October 30, 2026

1 Introduction

For a robot, the distance to surrounding objects must be known; otherwise, obstacles cannot be avoided and paths cannot be planned. Distance can be measured by depth sensors such as LiDAR, but these sensors are expensive. Normal cameras are cheap and are already mounted on almost every robot, so it would be very useful if depth could be estimated from a single image. This task is known as *monocular depth estimation* [1].

Today, very strong models such as Depth Anything V2 [2] have been developed for this task. However, these models are big and slow, so they cannot be run well on small robots, while lower accuracy is usually obtained with small, fast models. In addition, real depth maps are needed for training, and they can only be collected with a depth sensor and a lot of effort.

In this project, a small model (the *student*) is trained to learn from a big model (the *teacher*). This approach is called *knowledge distillation* [3]. Since any image can be labeled by the teacher for free, images without real depth can also be used for training the student. Distillation for depth has been studied before [4], and a foundation model was recently distilled in ZipDepth [5] with 14.1M images on two GPUs. In this project, the opposite end is studied: **the contribution of this project is a distillation recipe for very small students (1–4M parameters) when real depth is available for only a small part of the training images, together with a careful study of when the teacher helps and when it does not.** In the first tests (feasibility check in Section 3), no gain from the teacher was observed when real depth was available for every image; therefore, the focus is placed on the few-label setting, where a first gain of about 5% has already been observed over three random seeds.

2 Problem Statement

Task. A single RGB image is given, and a depth map, i.e. a distance value in meters for every pixel, is to be predicted.

Dataset. NYU Depth V2 [6] is used, in which indoor scenes were recorded with a Microsoft Kinect. The common training subset of 50,688 images (284 recordings from 249 rooms) [7] and the standard test set of 654 images are used. To simulate few labels, real depth is kept for only a part of the training images (1%, 5%, 25% or 100%), and the remaining images are treated as unlabeled. Since frames from the same room look very similar, 25 rooms are held out as a validation set, which is used for all tuning. The 654 test images are used only once, for the final results.

Expected results. The following hypotheses are made:

- with all labels, little is added by the teacher (this has already been observed in the feasibility check, Section 3);
- with few labels (1–5%), higher accuracy is expected when the teacher's outputs on the unlabeled images are also used than when only the labeled images are used, and this gap is expected to become small as the labeled part approaches 100%;
- the students are expected to be about 6 to 25 times smaller than the teacher (depending on their size) and to run in real time on a laptop GPU.

Evaluation. The standard depth metrics are used: absolute relative error (AbsRel), root mean squared error (RMSE) and threshold accuracy $\delta < 1.25$. For efficiency, the number of parameters, FLOPs and inference time are reported, and a comparison is made with published lightweight models such as FastDepth [11].

3 Technical Approach

Teacher. Depth Anything V2 Small [2] (24.8M parameters) is used as the teacher. It is run only once on all training images, and its outputs (*pseudo-labels*) are saved. Its output is relative inverse depth, which is correct only up to an unknown scale and shift, and this cannot be fixed on unlabeled images because no real depth is available for them. Therefore, a *scale-and-shift-invariant* loss [8] is used for the teacher term: both the student's prediction (as inverse depth) and the teacher output are normalized per image before they are compared. In this way, real depth is never used by the teacher term.

Student. A small encoder–decoder network is used as the student: a pretrained MobileNetV3 [9] or EfficientNet-B0 [10] encoder is combined with a light decoder similar to FastDepth [11]. Three sizes (about 1M, 3M and 4M parameters) are used, and the input size is set to 256×192.

Training loss.

$$\mathcal{L} \;=\; \lambda_1 \mathcal{L}_{\text{gt}} \;+\; \lambda_2 \mathcal{L}_{\text{teacher}} \;+\; \lambda_3 \mathcal{L}_{\text{edge}} \tag{1}$$

$\mathcal{L}_{\text{gt}}$ is the scale-invariant log loss [1] computed against the real depth; it is applied only to labeled images, and the real scale in meters is learned from it. $\mathcal{L}_{\text{teacher}}$ is the scale-and-shift-invariant L1 loss computed against the teacher; it is applied to all images, and the shape of the scene is learned from it. In $\mathcal{L}_{\text{edge}}$, the image gradients of the prediction and the teacher output are matched, so that sharp object borders are learned. Since the teacher term was observed to pull against the real-depth term in the first tests (see the feasibility check below), the teacher weight is increased from zero during the first 30% of training, very large updates are limited (gradient clipping), and λ_2 is tuned on the validation rooms.

Experiments. The same student is trained several times, and only one thing is changed each time. In the main setting, 5% labels are used.

ID	Setting
E1	Real depth only, on the labeled images (baseline)
E2	Real depth + teacher, on the labeled images only (is the teacher signal useful by itself?)
E3	Real depth on the labeled images + teacher on all images
E4	E3 + edge loss (full recipe)
E5	E1 vs. E4 with 1% / 5% / 25% / 100% labels
E6	E4 with student sizes of about 1M / 3M / 4M parameters (does the gain depend on capacity?)

The same number of training steps (about 63k at batch size 8) is used for all runs, and the saved model with the lowest validation error (best checkpoint) is kept. The main comparison (E1 vs. E4 at 5%) is repeated with three random seeds (different random starts), so that real differences can be separated from noise. Depth maps of success and failure cases are also shown.

Feasibility check. The plan was tested with short runs (1,500–2,000 steps) on a laptop (Apple M4 Pro, 24 GB). With all labels, no gain was obtained from the teacher (AbsRel 0.202 vs. 0.204), since the training depth maps are already complete (no missing pixels were found in 300 checked maps). With 5% labels (250 of 5,000 images), AbsRel was lowered from 0.221 to 0.211 (-4.5%) and $\delta < 1.25$ was raised from 0.608 to 0.648, and the teacher run was better for each of three seeds; this gain was reached only after the ramp-up and gradient clipping were added. A first test with the 1M and 4M students showed the same direction, with the smallest gain for the 1M student. The teacher needs about 40 ms per image (0.6 hours for all 50K images) and the 3M student 8.6 ms, so the 16 planned runs (about 4.6 hours each) need about 75 GPU hours, split between the laptop and Colab Pro.

4 Risks and Possible Failures

The main risks of the plan are summarized below, together with how they would be noticed and how they would be handled.

Risk	**Sign**	**Planned response**
The gain from the teacher shrinks or disappears at full scale (about 2,500 labels, longer training)	E3/E4 are not found better than E1 by more than the spread over seeds	The result is reported as a negative result, and the places where the teacher and the student disagree are analyzed (by depth range and image region). Metric pseudo-labels (teacher depth in meters) are also tried: teacher outputs are converted with one global scale fitted on the labeled images.
The real-depth term is opposed by the teacher term (already seen in the first tests)	The real-depth loss remains high; the predicted scale drifts away from the real one	The teacher weight is ramped up and gradients are clipped (this already helped in the first tests); λ_2 is reduced if needed and tuned on the validation rooms.
Differences are caused only by noise	Gaps are smaller than the standard deviation over three seeds	Only gaps larger than the spread are claimed; mean $\pm$ std is reported.

References

[1] D. Eigen, C. Puhrsch, and R. Fergus. Depth map prediction from a single image using a multi-scale deep network. In *NeurIPS*, 2014.

[2] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao. Depth Anything V2. In *NeurIPS*, 2024.

[3] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[4] Y. Wang, X. Li, M. Shi, K. Xian, and Z. Cao. Knowledge distillation for fast and accurate monocular depth estimation on mobile devices. In *CVPR Workshops*, 2021.

[5] F. Tosi, L. Bartolomei, M. Poggi, and S. Mattoccia. ZipDepth: Bringing lightweight zero-shot monocular depth anywhere, on any device. *arXiv:2607.08771*, 2026.

[6] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus. Indoor segmentation and support inference from RGBD images. In *ECCV*, 2012.

[7] I. Alhashim and P. Wonka. High quality monocular depth estimation via transfer learning. *arXiv:1812.11941*, 2018.

[8] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE TPAMI*, 2022.

[9] A. Howard et al. Searching for MobileNetV3. In *ICCV*, 2019.

[10] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

[11] D. Wofk, F. Ma, T.-J. Yang, S. Karaman, and V. Sze. FastDepth: Fast monocular depth estimation on embedded systems. In *ICRA*, 2019.